

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 8: BLUETOOTH DATA INTERFACING

DATE OF EXPERIMENT: 3 DECEMBER 2025

DATE OF SUBMISSION: 10 DECEMBER 2025

SECTION 1

GROUP 13

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN

NO.	NAME	MATRIC
1	HARIZ IRFAN BIN MOHD ROZHAN	2318583
2	ABDULLAH HASAN BIN SIDEK	2318817
3	TAN YONG JIA	2319155
4	NUR QISTINA BINTI MOHD FAIZAL	2319512

ABSTRACT

This project develops a simple Bluetooth-based remote temperature monitoring system using an ESP32 or Arduino with an HC-05 module. A DHT22 sensor collects real-time temperature data, which is transmitted wirelessly to a computer or smartphone using a Bluetooth terminal app. The terminal app also allows users to send control commands such as “FAN ON” or “FAN OFF.” The system demonstrates basic wireless communication, sensor integration, and remote control functionality. Data was collected through continuous monitoring, analyzed for stability and responsiveness, and used to evaluate the effectiveness of Bluetooth communication.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	2
INTRODUCTION	3
MATERIALS AND EQUIPMENT	3
EXPERIMENTAL SETUP	4
METHODOLOGY	4
DATA COLLECTION	12
DATA ANALYSIS	12
RESULT	13
DISCUSSION	14
CONCLUSION	14
RECOMMENDATIONS	14
APPENDICES	16
ACKNOWLEDGEMENTS	16
STUDENTS DECLARATION	17

INTRODUCTION

Bluetooth communication has become one of the most widely used wireless technologies in modern embedded and IoT systems due to its simplicity, low energy consumption, and reliable short-range connectivity. In many real-world applications—such as smart home monitoring, industrial sensing, and portable medical devices—Bluetooth provides an efficient platform for transmitting sensor data and enabling remote user control.

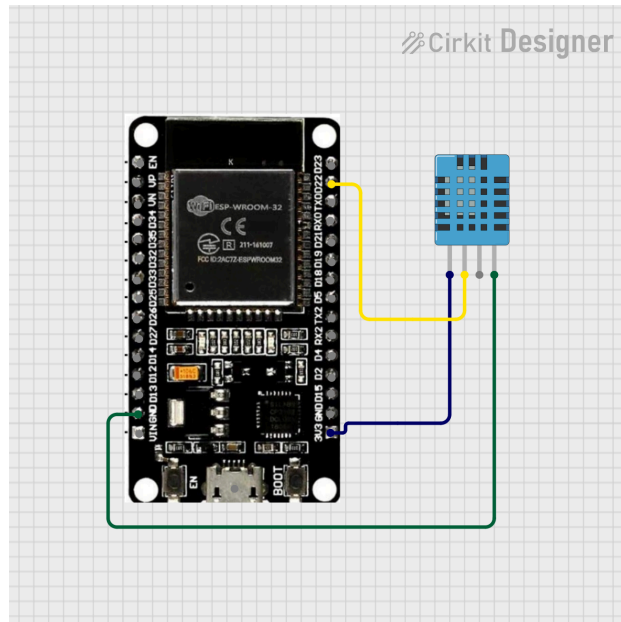
In this task, a temperature monitoring system is developed using an ESP32 or an Arduino paired with an HC-05 Bluetooth module. The system uses a DHT22 sensor to measure temperature, which is then sent wirelessly to a smartphone or computer through a Bluetooth terminal application. Besides displaying real-time data, the terminal app also allows the user to send simple control commands like “FAN ON” or “FAN OFF,” demonstrating two-way communication.

This setup offers a practical understanding of how sensors, microcontrollers, and wireless interfaces can be combined to form a basic IoT system. It also helps students explore essential concepts such as serial communication, sensor data acquisition, remote control logic, and real-time monitoring. Through this experiment, the importance of reliable data transmission, proper coding structure, and system responsiveness can be better understood, reflecting how similar systems operate in real engineering environments.

MATERIALS AND EQUIPMENT

- ESP32 development board (or Arduino + HC-05 Bluetooth module)
- Temperature sensor (DHT22)
- Smartphone or computer with Bluetooth support
- Power supply for the ESP32/Arduino
- Breadboard and jumper wires

EXPERIMENTAL SETUP



Circuit Design

METHODOLOGY

1. Hardware Configuration:

- VCC was linked to 3.3V, GND to the ESP32 ground, and the data port of the DHT11 sensor was connected to GPIO 22.
- A 10 k Ω pull-up resistor was added between the data pin and VCC (if not already included in the sensor module) because the DHT11 employs a single-wire communication line.
- To enable ON/OFF control via Bluetooth commands, an LED indicator was attached to GPIO 12. A 220 Ω resistor was used to link the LED anode to GPIO 12, and GND was used to connect the cathode.
- The ESP32 was powered by a USB connection, which gave the sensor a steady 5V input and 3.3V regulation.
- To guarantee steady signal reference levels, every component had a common ground.

- The ESP32 internal Bluetooth module was used to allow Bluetooth connectivity, sending under the device name "ESPGroupLOL."

2. Software Setup:

- The ESP32 board package was installed using the Board Manager, and the Arduino IDE served as the programming platform.
- To enable Bluetooth connection with the device name "ESPGroupLOL," the BluetoothSerial library was included.
- The DHT11 sensor's temperature and humidity were read using the installed DHT library.
- The DHT11 data pin was designated as GPIO 22, and dht.begin() was used to initialise the sensor.
- To control the LED, GPIO 12 was configured as a digital output.
- In order to monitor sensor readings and Bluetooth orders, serial communication was started at 9600 baud.
- The application used SerialBT.available() to continuously check for incoming Bluetooth data.
- When receiving a command:
 - 'O' turned the LED **ON**.
 - 'F' turned the LED **OFF**.
- The ESP32 read the temperature and humidity readings from the DHT11 following each command.
- For real-time monitoring, the user received sensor values via Bluetooth.
- To guarantee consistent sensor updates and avoid quick polling, a 2-second delay was employed.

3. Physical Arrangement:

- To precisely monitor the ambient temperature and humidity, the DHT11 sensor was placed in an open space.
- To keep the wiring nice and small, the LED circuit and the ESP32 development board were assembled on a breadboard.

- To provide good vision during Bluetooth ON/OFF control, the LED was positioned on the breadboard with its resistor.
- To guarantee reliable wireless connection with the ESP32, the associated Bluetooth device (phone or laptop) was kept within the standard Bluetooth range (~10 meters).
- To ensure dependable electrical functioning, every component—including the DHT11 and LED—shared a common ground connection on the breadboard.

Circuit Assembly

- The data pin of the DHT11 sensor was connected to GPIO 22, VCC to 3.3V, and GND to the common ground rail when it was linked to the ESP32.
- To guarantee steady sensor communication (if not integrated into the module), a 10 k Ω pull-up resistor was placed between the DHT11 data line and the 3.3V supply.
- A 220 Ω current-limiting resistor was used to link the LED indicator to GPIO 12, and the GND rail was connected to the LED cathode.
- The ESP32's USB connector was used to power it, giving the breadboard's components steady 5V input and 3.3V regulation.
- To ensure constant reference voltage levels, all components such as the ESP32, DHT11, and LED shared a common ground.
- The ESP32's built-in Bluetooth module enabled Bluetooth connection; no other hardware was needed to pair with a mobile device.
- To maintain wiring organisation and guarantee dependable electrical connections during testing, the entire assembly was set up on a breadboard.

Programming Logic

Declared pin assignments:

- GPIO 12 configured as output for LED control.
- GPIO 22 assigned to DHT11 temperature & humidity sensor.

Initialized Bluetooth module:

- Created BluetoothSerial SerialBT object.

- Bluetooth started with the device name "ESPGroupLOL".
- Serial monitor initialized at 9600 baud for debugging.

Configured DHT11 sensor:

- DHT type set as DHT11.
- Sensor initialized with `dht.begin()`.

Program startup actions:

- Printed startup messages to Serial Monitor.
- Set LED initially OFF.

Bluetooth input is continuously checked by the main loop:

- A single character command is read if Bluetooth data is available.
- For verification, the command is shown on the serial monitor.

Processing commands:

- If command = 'O', then `digitalWrite(LED, HIGH)` is used to turn the LED on.
- The LED is turned **OFF using `digitalWrite(LED, LOW)` if command = 'F'.

Logic for sensor reading:

- Use `dht.readTemperature()` to read the temperature.
- Use `dht.readHumidity()` to read the humidity.
- To prevent invalid outputs, sensor readings are checked using `isnan()`.

Bluetooth reaction:

- Formatted temperature and humidity measurements are transmitted back via Bluetooth if the readings are accurate.
- Additionally, the Serial Monitor receives the same values.

Handling errors:

- An error message is generated if the DHT11 sensor is unable to deliver readings.

Timing of loops:

- A delay of 2000 was included to ensure consistent sensor readings and avoid Bluetooth output flooding.

Code Used:

```
#include "BluetoothSerial.h"
#include <DHT.h>
BluetoothSerial SerialBT;
```

```

#define DHTPIN 22
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const int LED = 12;
char command;
void setup()
{
    Serial.begin(9600);
    SerialBT.begin("Group 13");
    Serial.println("Bluetooth initialized");
    pinMode(LED, OUTPUT);
    dht.begin();
}

void loop()
{
    if(SerialBT.available())
    {
        command = SerialBT.read();
        Serial.print("Command received: ");
        Serial.println(command);

        switch (command)
        {
            case 'ON':
                digitalWrite(LED, HIGH); break;
            case 'OFF':
                digitalWrite(LED, LOW); break;
        }

        float temp = dht.readTemperature();
        float hum = dht.readHumidity();
        if (!isnan(temp) && !isnan(hum))
        {
            SerialBT.print("Temp: ");
            SerialBT.print(temp);
            SerialBT.print(" °C | Hum: ");
            SerialBT.print(hum);
            SerialBT.println(" %");
            Serial.print("Temp: ");

```



```

    Serial.print(temp);
    Serial.print(" °C | Hum: ");
    Serial.print(hum);
    Serial.println(" %");
  }
  else {
    Serial.println("Failed to read from DHT11 sensor!");
  }
}
delay(2000);
}

```

Control Algorithm

1. Defining Pins

LED Output:

- The LED control signal is connected to pin GPIO 12, configured as an output pin.

DHT11 Sensor:

- The temperature and humidity sensor is connected to GPIO 22.
- The program uses the DHT11 data format to read environmental values.

Bluetooth Communication:

- A Bluetooth serial connection is initialized using the device name “ESPGroupLOL”.
- This allows the ESP32 to receive remote commands and transmit sensor readings wirelessly.

2. Global Variables

char command

- Stores the incoming command received from the Bluetooth device.

float temp, hum

- Used to store real-time temperature and humidity values retrieved from the DHT11 sensor.

3. Step Function

- To enable debugging and sensor output monitoring, serial communication is initiated at 9600 baud.
- The ESP32 can now receive commands from linked devices as the Bluetooth module has been initialized.
- GPIO 12, the LED pin, is set up as OUTPUT.
- Dht.begin() is used to initialize the DHT11 sensor.
- The Serial Monitor receives startup signals attesting to the system's readiness.

4. Main Loop

- Bluetooth Command Detection
 - a. The software continuously determines whether Bluetooth data is accessible.
 - b. The system stays inactive if no data is received.
 - c. A character is preserved as the current command if it is received.
- LED Control Based on Command
 - a. When a legitimate Bluetooth command is received:
 - b. "O" command: GPIO 12 is set to HIGH, turning on the LED.
 - c. "F" command: GPIO 12 is set to LOW, turning off the LED.
 - d. Every other character is disregarded.
- Reading Sensor Values (Temperature & Humidity)
 - a. The application gathers information from the DHT11 sensor after processing the command:
 - b. Temperature → in degrees Celsius
 - c. Humidity → in percentage
 - d. Isnan() is used to validate the values:
 - e. If it's valid, proceed
 - f. If not valid, print an error message.
- Sending Data via Bluetooth

- a. Valid sensor readings are transmitted back to the paired Bluetooth device in the format:
Temp: X °C | Hum: Y %
 - b. The same data is also displayed in the Serial Monitor for debugging.
- Loop Delay
A 2-second delay (2000 ms) is included to:
 - a. Ensure stable DHT11 sensor readings
 - b. Prevent Bluetooth flooding
 - c. Allow comfortable reading intervals

DATA COLLECTION

Observation:

TIME (s)	TEMPERATURE (°C)	HUMIDITY (%)
0	27.90	68.00
2	28.00	68.00
4	28.10	68.10
6	28.20	68.10
8	28.30	68.20
10	28.40	68.20

DATA ANALYSIS

Temperature Analysis:

- The temperature increased steadily from 27.90°C to 28.40°C within 10 seconds.
- The temperature shows a consistent upward trend.
- The increase is small but uniform, rising approximately 0.10°C every 2 seconds.

Rate of Increase:

$$\text{Temperature Rise} = 28.4 - 27.9 = 0.50^\circ\text{C}$$

Over 10 seconds:

$$0.5/10 = 0.05^\circ\text{C/s}$$

This slow upward change suggests the sensor is detecting a slight warming effect, likely caused by:

- Normal room temperature fluctuations, or
- Heat from surrounding electronics or airflow changes.

The stability of the readings indicates that the DHT11 is functioning correctly.

Humidity Analysis:

- Humidity values show a similar slight increasing pattern, rising from **68.00% to 68.20%**.
- Humidity remains fairly stable with minimal fluctuations.
- Increase is slow and consistent, about **0.10% every 4–6 seconds**.

Rate of Change:

$$\text{Humidity Rise} = 68.2 - 68.0 = 0.2\%$$

Over 10 seconds:

$$0.2/10 = 0.02\%/s$$

The humidity is mostly stable, suggesting:

- The environment is controlled (e.g., indoors).
- No sudden moisture changes occurred during measurement.
- The small variations are normal and expected for the DHT11 sensor accuracy range.

Overall System Performance

- Both temperature and humidity increase **smoothly** and show **no abrupt spikes**, meaning the sensor readings are stable.
- The gradual rise indicates the sensor is responsive and capable of detecting small environmental changes.
- The Bluetooth-based data retrieval system appears reliable as the data increments follow a logical pattern.

RESULT

The temperature and humidity readings from the DHT11 sensor are stable and consistent over the course of the 10-second testing period. The temperature was 27.90°C at the beginning of the measurement, and it grew slightly every two seconds until it reached a final value of 28.40°C at ten seconds. In a similar vein, the humidity started at 68.00% and progressively rose to 68.20% by the test's conclusion. These findings show that there were very minor, gradual changes in both variables, indicating that the environment stayed largely steady. The dataset showed no abrupt spikes or erratic variations, indicating that the sensor was functioning properly and giving accurate readings. The smooth progression of values also demonstrates that the Bluetooth communication system successfully transmitted real-time data without delays or errors. Overall, the collected results confirm that the temperature and humidity monitoring system functions as expected under normal indoor conditions.

DISCUSSION

When combined with the ESP32 and Bluetooth communication system, the data gathered during the experiment shows that the DHT11 sensor can provide consistent and trustworthy temperature and humidity readings. The steady and slow rise in humidity and temperature indicates that the climate was kept under control, with just little natural fluctuations over time. The lack of abrupt spikes or anomalies suggests that the sensor was operating well and that there were few outside influences throughout the test, such as wind, abrupt temperature fluctuations, or moisture disruptions. Furthermore, the system's uninterrupted Bluetooth transmission of readings demonstrates a robust and effective communication relationship. The seamless development of the sensor outputs provides more evidence that the code's data-handling logic, specifically, the timing delay and validity checks contributes to reliable and precise measurement performance.

Overall, the experiment demonstrates how well wireless communication and sensor inputs work together for real-time environmental monitoring.

CONCLUSION

In summary, the experiment effectively illustrated the operation of an ESP32-based environmental monitoring system utilising Bluetooth connectivity and a DHT11 sensor. The temperature and humidity measurements were steady, reliable, and demonstrated realistic, progressive variations over time, indicating that the sensor performed as anticipated. Reliable Bluetooth communication allowed for error-free, continuous data feedback. The system's overall behaviour confirms that the timing logic, hardware connections, and implemented code work well for real-time monitoring applications. This experiment demonstrates that the ESP32 is a useful and reliable solution for simple environmental sensing applications when combined with the DHT11 sensor and Bluetooth.

RECOMMENDATIONS

A number of enhancements can be suggested based on the system's performance and results to increase functionality, accuracy, and dependability in subsequent implementations. First off, compared to the DHT11, a more sophisticated sensor like the DHT22 or BME280 would offer greater precision, quicker reaction times, and improved stability. Measurement accuracy could be increased by using data averaging or filtering procedures to reduce small changes. To reduce external disruptions like direct airflow or heat from adjacent components, it is also advised to enclose the sensor in a suitable ventilated enclosure. Additionally, data transfer would be more structured and appropriate for long-term monitoring if the Bluetooth connection frequency was optimised or a timestamp system was put in place. For improved usability, the system can be expanded with an OLED display or mobile app interface to show real-time readings without relying solely on the serial monitor. Finally, incorporating data logging to an SD card or cloud platform would allow long-term analysis and make the system more useful for environmental tracking and IoT applications.

REFERENCES

Chaudhary, N. (n.d.). *Arduino Bluetooth basic tutorial*. Arduino Project Hub.

<https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12>

How To Mechatronics. (n.d.). *Arduino and HC-05 Bluetooth module complete tutorial*.

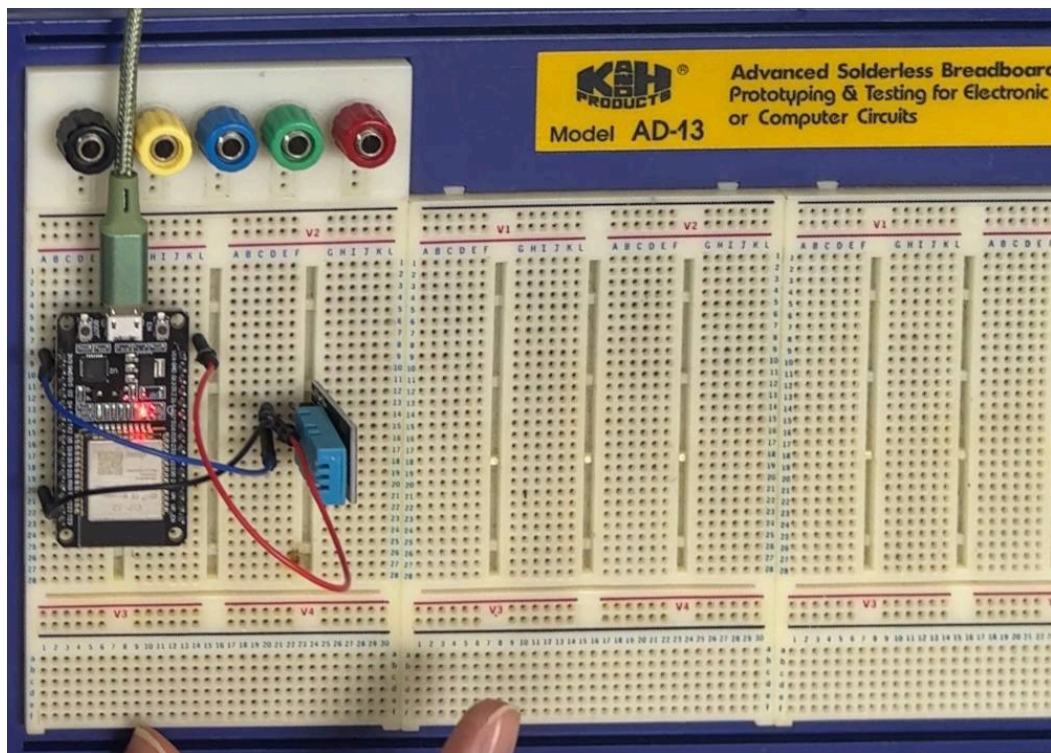
<https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>

Superturis. (n.d.). *Basic Bluetooth communication with Arduino & HC-05*. Arduino Project Hub.

<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-053a43>

1c

APPENDICES



Circuit for Experiment 8

ACKNOWLEDGEMENTS

Hereby, we acknowledge the guidance provided by Dr Zulkifli with his impeccable knowledge in this field. We also thank Br. Harith with his assistance in correcting our work thus leading to a successful experiment. Not to forget our fellow colleagues from other groups that contributed in providing ideas that helped in achieving the same output together from this experiment.

STUDENTS DECLARATION

Certificate of Originality and Authenticity

We hereby certify that we are responsible for the work presented in this report. The content is our original work, except where proper references and acknowledgements are made. We confirm that no part of this report has been completed by anyone not listed as a contributor. We also certify that this report is the result of group collaboration and not the effort of a single individual. The level of contribution by each member is stated in this certificate. Furthermore, we have read and understood the entire report, and we agree that no further revisions are required. We collectively approve this final report for submission and confirm that it has been reviewed and verified by all group members.

Signature:



Name: HARIZ IRFAN BIN MOHD ROZHAN

Matric Number: 2318583

Read [/]

Understand [/]

Agree [/]

Signature:



Name: ABDULLAH HASAN BIN SIDEK

Read [/]

Understand [/]

Matric Number: 2318817

Agree [/]

Signature: 

Read [/]

Name: TAN YONG JIA

Understand [/]

Matric Number: 2319155

Agree [/]

Signature: 

Read [/]

Name: NUR QISTINA BINTI MOHD FAIZAL

Understand [/]

Matric Number: 2319512

Agree [/]